

Curso: Programación Avanzada

Guía: 06

Título: Casos de Uso – La Capa de Aplicación

Duración estimada: 90 minutos

Docente: Christian Andrés Candela

CASOS DE USO: LA CAPA DE APLICACIÓN

"Los casos de uso no contienen reglas del negocio: contienen decisiones sobre **cuándo** ejecutar esas reglas."

OBJETIVO

En las guías anteriores construimos:

- entidades y value objects (Guía 04),
- agregados y reglas de negocio (Guía 05).
- pruebas unitarias del dominio (Guía 05.1).

Ahora tenemos un **dominio rico** que sabe qué está permitido y qué no, y que adicionalmente sabemos que funciona correctamente.

Pero surge una pregunta:

"Ya tengo un dominio rico... ¿cómo se usa desde afuera sin dañarlo?"

Esta guía responde esa pregunta.

El objetivo **no es agregar más lógica**, sino aprender a:

- orquestar el dominio desde una capa externa,

- coordinar objetos sin tomar decisiones de negocio,
- preparar el sistema para exponerlo (sin hacerlo aún).

CONTEXTUALIZACIÓN TEÓRICA

¿Qué es un Caso de Uso?

Un **caso de uso** representa una acción que alguien quiere realizar sobre el sistema.

No es una definición UML formal. Es una intención concreta:

- Registrar una solicitud
- Asignar responsable a una solicitud
- Cerrar una solicitud
- Consultar solicitudes por estado

Cada una de estas acciones tiene un **actor** (quien la inicia) y un **propósito** (qué espera lograr).

Un Caso de Uso NO es

- ✗ Un servicio de negocio renombrado
- ✗ Una clase que valida reglas complejas
- ✗ Un lugar para poner `if` de negocio
- ✗ Un CRUD con otro nombre
- ✗ Una excusa para introducir frameworks

Si una clase toma decisiones del negocio, **no es un caso de uso**.

Un Caso de Uso SÍ es

Un **orquestador** que:

1. recibe una intención (parámetros simples),
2. obtiene o crea objetos del dominio,
3. invoca comportamiento del dominio,
4. devuelve un resultado.

Nada más.

El Rol de Cada Capa

Capa	Responsabilidad
Dominio	Reglas, invariantes, comportamiento
Caso de Uso	Orquestrar el dominio
Infraestructura	Detalles técnicos (después)

El dominio **sabe qué está permitido**.
El caso de uso **sabe cuándo y en qué orden**.

¿De dónde vienen los objetos del dominio?

Observemos el paso 2 de un caso de uso: *"obtener o crear objetos del dominio"*. Esto plantea una pregunta fundamental:

Si el caso de uso necesita una `Solicitud` para cerrarla... ¿de dónde la obtiene?

En el mundo real, esas solicitudes estarán almacenadas en algún lugar: una base de datos, un archivo, un servicio externo. Pero nuestro caso de uso **no debería saber** dónde ni cómo están almacenadas. Eso es un detalle técnico.

Necesitamos un mecanismo que permita al caso de uso **pedir objetos del dominio sin conocer su origen**. Ese mecanismo es el **Repositorio**.

El Patrón Repository (DDD)

En el contexto de Domain-Driven Design, un **Repositorio** no es una clase de acceso a datos. Es un concepto más abstracto:

Un repositorio simula una **colección en memoria** de objetos del dominio.

Desde la perspectiva del caso de uso, un repositorio es simplemente un lugar donde se pueden:

- **buscar** objetos del dominio (por identificador, por criterio),
- **guardar** objetos nuevos o modificados.

El repositorio se expresa como una **interfaz** (un contrato) que define **qué operaciones** necesita el dominio, sin decir **cómo** se implementan:

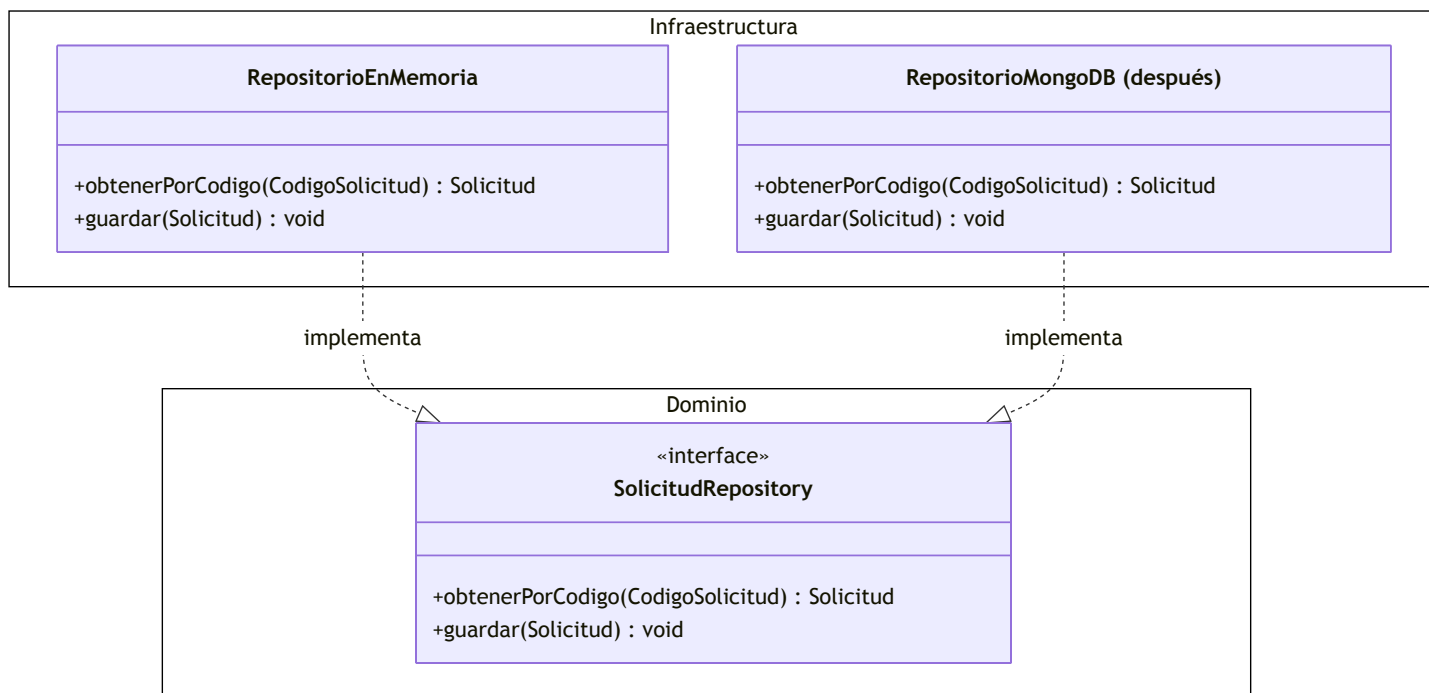
```
// Esto es un contrato, no una implementación
public interface SolicitudRepository {
    Solicitud obtenerPorCodigo(CodigoSolicitud codigo);
    void guardar(Solicitud solicitud);
}
```

Esta interfaz no menciona bases de datos, SQL, MongoDB ni ninguna tecnología. Solo dice: *"necesito poder obtener solicitudes y guardarlas"*.

Interfaz en el dominio, implementación en infraestructura

Un principio fundamental de la arquitectura limpia es la **Inversión de Dependencias**:

Las capas internas (dominio) definen **qué necesitan**. Las capas externas (infraestructura) definen **cómo se cumple**.



La interfaz del repositorio **vive en el dominio** porque es el dominio quien necesita esas operaciones. La implementación concreta (qué base de datos usar, cómo conectarse) vive en infraestructura y llegará en guías posteriores.

¿Por qué en memoria por ahora?

En esta guía usaremos implementaciones **en memoria** (con `HashMap`). Esto tiene ventajas pedagógicas y técnicas fundamentales:

1. **Validar el diseño sin atarse a tecnología:** podemos probar que nuestros casos de uso funcionan correctamente sin instalar ni configurar ninguna base de datos.
2. **Enfoque en la lógica:** el estudiante se concentra en la orquestación, no en la configuración de infraestructura.
3. **Intercambiabilidad:** cuando lleguemos a la Guía 10, reemplazaremos la implementación en memoria por una real (MongoDB), **sin modificar ni una línea de los casos de uso.**

La persistencia real vendrá después. Por ahora, lo importante es que la **arquitectura** ya está preparada para ella.

ESTRUCTURA DE UN CASO DE USO

Forma mental

Un caso de uso típico sigue esta estructura:

1. Recibir intención (parámetros simples)
↓
2. Obtener o crear objetos del dominio
↓
3. Invocar comportamiento del dominio
↓
4. Devolver resultado

Ejemplo: Cerrar una Solicitud

```
public class CerrarSolicitudUseCase {  
  
    private final SolicitudRepository repository;  
  
    public CerrarSolicitudUseCase(SolicitudRepository repository) {  
        this.repository = repository;  
    }  
  
    public void ejecutar(CodigoSolicitud codigo) {  
        Solicitud solicitud = repository.obtenerPorCodigo(codigo);  
        solicitud.cerrar();  
        repository.guardar(solicitud);  
    }  
}
```

Observa:

- ❌ No hay reglas de negocio aquí
- ❌ No hay validaciones complejas
- ❌ No hay decisiones sobre qué está permitido
- ✅ Solo coordinación

La validación de si la solicitud **puede** cerrarse vive en el dominio:

```
// En el dominio (Guía 05)  
public class Solicitud {  
  
    public void cerrar() {  
        if (this.responsable == null) {  
            throw new ReglaDominioException("No se puede cerrar sin responsable");  
        }  
        if (this.estado != EstadoSolicitud.ATENDIDA) {  
            throw new ReglaDominioException("Solo se puede cerrar una solicitud atendid");  
        }  
        this.estado = EstadoSolicitud.CERRADA;  
    }  
}
```

El caso de uso **no repite** esa lógica. Solo coordina.

LO QUE NO HACE UN CASO DE USO

Un caso de uso:

- ✗ no valida estados complejos
- ✗ no decide si algo está permitido
- ✗ no contiene `if` de negocio
- ✗ no conoce HTTP, JSON ni bases de datos
- ✗ no sabe de seguridad ni autenticación

Esas responsabilidades pertenecen a otras capas.

LA INTERFAZ DEL REPOSITORIO

Los casos de uso dependen de **interfaces**, no de implementaciones.

```
public interface SolicitudRepository {  
  
    Solicitud obtenerPorCodigo(CodigoSolicitud codigo);  
  
    void guardar(Solicitud solicitud);  
  
    List<Solicitud> obtenerPorEstado(EstadoSolicitud estado);  
}
```

Esta interfaz:

- vive en el **dominio** (o en la capa de aplicación),
- será implementada después en infraestructura,
- permite que los casos de uso funcionen sin base de datos real.

Por ahora, podemos usar una implementación en memoria para probar:

```
public class SolicitudRepositoryEnMemoria implements SolicitudRepository {

    private final Map<CodigoSolicitud, Solicitud> solicitudes = new HashMap<>();

    @Override
    public Solicitud obtenerPorCodigo(CodigoSolicitud codigo) {
        Solicitud solicitud = solicitudes.get(codigo);
        if (solicitud == null) {
            throw new SolicitudNoEncontradaException(codigo);
        }
        return solicitud;
    }

    @Override
    public void guardar(Solicitud solicitud) {
        solicitudes.put(solicitud.getCodigo(), solicitud);
    }

    @Override
    public List<Solicitud> obtenerPorEstado(EstadoSolicitud estado) {
        return solicitudes.values().stream()
            .filter(s -> s.getEstado() == estado)
            .toList();
    }
}
```


MÁS EJEMPLOS DE CASOS DE USO

Crear una Solicitud

```
public class CrearSolicitudUseCase {

    private final SolicitudRepository repository;
    private final GeneradorCodigo generadorCodigo;

    public CrearSolicitudUseCase(SolicitudRepository repository,
                                GeneradorCodigo generadorCodigo) {
        this.repository = repository;
        this.generadorCodigo = generadorCodigo;
    }

    public Solicitud ejecutar(TipoSolicitud tipo, String descripcion, Usuario solicitante) {
        CodigoSolicitud codigo = generadorCodigo.generar();

        Solicitud solicitud = Solicitud.crear(codigo, tipo, descripcion, solicitante);

        repository.guardar(solicitud);

        return solicitud;
    }
}
```

Asignar Responsable

```
public class AsignarResponsableUseCase {

    private final SolicitudRepository solicitudRepository;
    private final UsuarioRepository usuarioRepository;

    public AsignarResponsableUseCase(SolicitudRepository solicitudRepository,
                                     UsuarioRepository usuarioRepository) {
        this.solicitudRepository = solicitudRepository;
        this.usuarioRepository = usuarioRepository;
    }

    public void ejecutar(CodigoSolicitud codigo, String responsableId) {
        Solicitud solicitud = solicitudRepository.obtenerPorCodigo(codigo);
        Usuario responsable = usuarioRepository.obtenerPorId(responsableId);

        solicitud.asignarResponsable(responsable);

        solicitudRepository.guardar(solicitud);
    }
}
```

Consultar Solicitudes por Estado

```
public class ConsultarSolicitudesPorEstadoUseCase {

    private final SolicitudRepository repository;

    public ConsultarSolicitudesPorEstadoUseCase(SolicitudRepository repository) {
        this.repository = repository;
    }

    public List<Solicitud> ejecutar(EstadoSolicitud estado) {
        return repository.obtenerPorEstado(estado);
    }
}
```

En todos los casos, el patrón es el mismo:

1. Recibir parámetros

2. Obtener objetos del dominio
3. Invocar comportamiento (si aplica)
4. Devolver resultado

ESTRUCTURA DEL PROYECTO

Los casos de uso viven en un paquete separado del dominio:

```
src/main/java/co/edu/uniquindio/solicitudes/  
├── domain/  
│   ├── entity/  
│   │   ├── Solicitud.java  
│   │   └── Usuario.java  
│   ├── valueobject/  
│   │   ├── CodigoSolicitud.java  
│   │   ├── EstadoSolicitud.java  
│   │   └── Prioridad.java  
│   ├── repository/  
│   │   ├── SolicitudRepository.java  
│   │   └── UsuarioRepository.java  
│   └── exception/  
│       └── ReglaDominioException.java  
└── application/  
    └── usecase/  
        ├── CrearSolicitudUseCase.java  
        ├── AsignarResponsableUseCase.java  
        ├── CerrarSolicitudUseCase.java  
        └── ConsultarSolicitudesPorEstadoUseCase.java
```

Observa:

- domain/ contiene las reglas
- application/usecase/ contiene la coordinación
- No hay dependencias de Spring, REST ni bases de datos

ACTIVIDAD PRÁCTICA

Objetivo

Implementar casos de uso que utilicen el dominio ya construido en las guías anteriores.

Instrucciones

1. Cree los siguientes casos de uso:

- `CrearSolicitudUseCase`
- `AsignarResponsableUseCase`
- `CambiarEstadoUseCase`
- `CerrarSolicitudUseCase`
- `ConsultarSolicitudesPorEstadoUseCase`

2. Cada caso de uso debe:

- vivir en el paquete `application.usecase`
- depender de interfaces (repositorios), no de implementaciones
- invocar comportamiento del dominio

3. Cree las interfaces de repositorio necesarias en el paquete `domain.repository`

4. Cree una implementación en memoria de cada repositorio para probar los casos de uso

5. Importante: El dominio **no se modifica** en esta guía. Si necesita agregar comportamiento, debe hacerlo en las entidades existentes.

PRECAUCIONES Y RECOMENDACIONES

1. **No dupliques validaciones:** Si el dominio ya valida algo, no lo repitas en el caso de uso.
2. **Parámetros simples:** Los casos de uso reciben tipos simples o value objects, no DTOs complejos (eso viene después).
3. **Un caso de uso = una intención:** No mezcles múltiples acciones en un solo caso de uso.
4. **Prueba manualmente:** Crea una clase `Main` temporal para probar los casos de uso con repositorios en memoria.

PRÓXIMA ACTIVIDAD

En la **Guía 07: Diseño de APIs desde el Dominio** aprenderemos a:

- mapear casos de uso a endpoints REST
- diseñar DTOs para entrada y salida
- documentar la API con OpenAPI

Los casos de uso que construiste hoy serán los que expondrás mañana.

REFERENCIAS BIBLIOGRÁFICAS

1. Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley.
2. Vernon, V. (2013). *Implementing Domain-Driven Design*. Addison-Wesley.
3. Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
4. Cockburn, A. (2000). *Writing Effective Use Cases*. Addison-Wesley.